

RASHMITHA ETTADI
Assignment3
REPORT

Outline of all classes:

Customer class:

Attributes:

id (int)

name (string)

email (string)

password (string)

shippingAddress (string)

Methods:

register(): Creates a new customer account.

login(): Authenticates a customer and allows them to access the system.

addToCart(product): Adds a product to the customer's shopping cart.

removeFromCart(product): Removes a product from the customer's shopping cart.

viewCart(): Allows the customer to view the items in their shopping cart.

purchase(): Processes the customer's order and charges their payment method.

ShoppingCart class:

Attributes:

items (list of Product) - This could be an array or another data structure that stores information on the products in the cart.

quantity (list of int) - This could be an array or another data structure that stores the quantity of each product in the cart. This corresponds to the items list.

totalPrice (float)

Methods:

addItem(product, quantity): Adds a product to the shopping cart and keeps track of the quantity.

removeItem(product): Removes a product from the shopping cart.

checkout(): Initiates the checkout process and prepares the order for purchase.

Product class:

Attributes:

id (int)

customerID (int) - This might be a foreign key referencing the Customer class

name (string)

price (float)

description (string)

Methods:

addToStock(quantity): Increases the available stock level of the product by the specified quantity.

removeFromStock(quantity): Decreases the available stock level of the product by the specified quantity.

updatePrice(new_price): Updates the price of the product to the provided new price.

updateDescription(new_description): Updates the textual description of the product with the new description.

Order class:

Attributes:

id (int)

customerID (int) - This might be a foreign key referencing the Customer class

orderDate (date) - This could be a string or another data structure depending on the programming language.

products (list of Product) - This could be an array or another data structure that stores information on the products in the order.

Methods: (possibly incomplete)

createOrder(): Creates a new order.

Administrator class:

Attributes:

id (int)

name (string)

email (string)

Methods:

createUser(customer): Creates a new customer account. Might be used for creating admin users as well.

updateUser(customer): Updates an existing customer's information.

removeUser(customer): Removes a customer from the system.

getProducts(): Retrieves all products from the system.

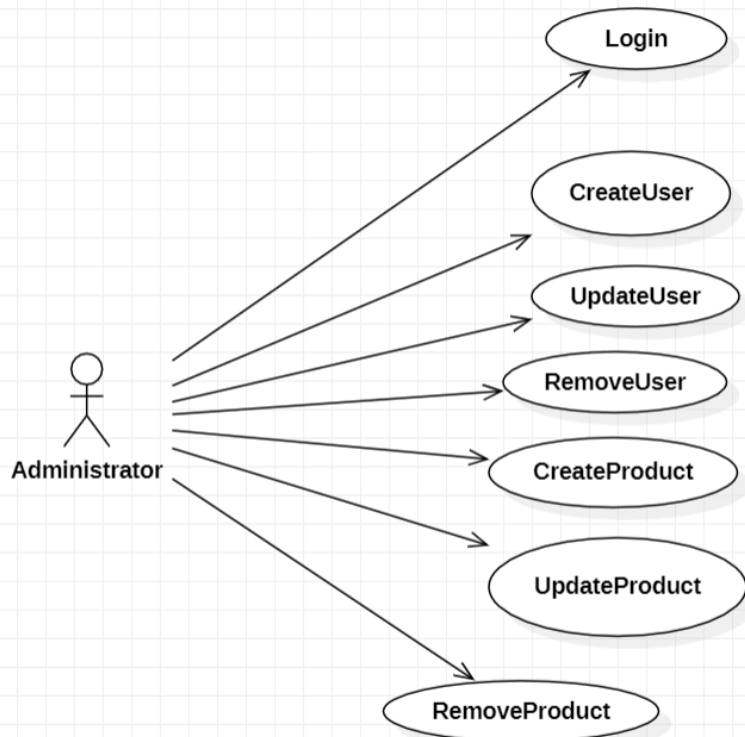
createProduct(product): Creates a new product.

updateProduct(product): Updates an existing product's information.

removeProduct(product): Removes a product from the system.

processOrder(order): Processes an order and updates the inventory levels.

Use case diagram for Administrator



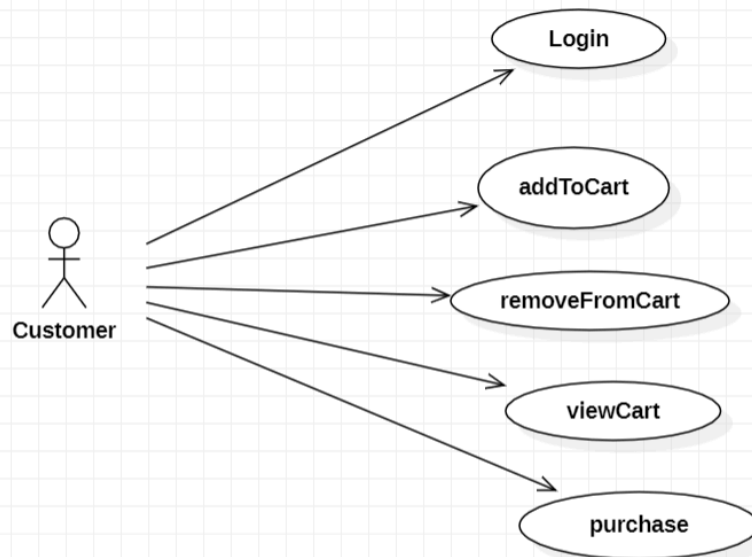
Actors:

Administrator

Use Cases:

Administrator: Create User, Update User, Remove User, View Products, Create Product, Update Product, Remove Product, Process Order

UseCase Diagram for Customer



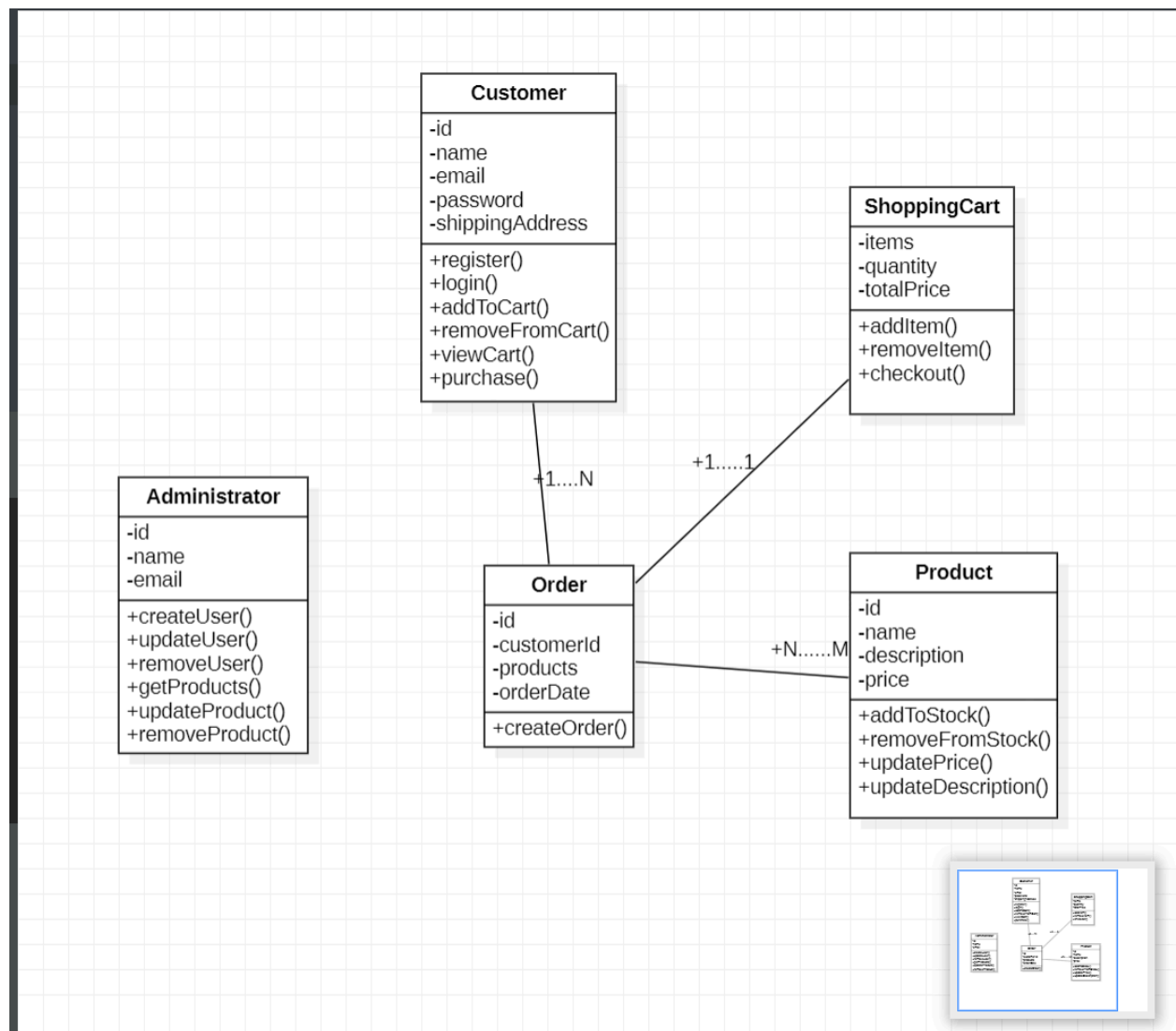
Actors:

Customer

Use Cases:

Customer: Create Account, Login, View Products, Add to Cart, Remove from Cart, View Cart, Purchase

Class Diagram



Domain Model based on the Class Diagram

The provided class diagram represents a portion of a possible domain model for an e-commerce system. Here's a breakdown of the domain concepts and their relationships:

Entities:

Customer: Represents a customer who can register, login, browse products, add them to a shopping cart, and purchase them.

Attributes:

id (unique identifier)

name

email (unique identifier for login)

password

shippingAddress (optional)

Product: Represents a product sold on the platform.

Attributes:

id (unique identifier)

customerID (optional - might indicate a vendor if implemented) - This could be a foreign key referencing the Customer class if the system allows vendors to create products.

name

price

description

ShoppingCart: Represents a temporary storage for products a customer intends to purchase.

Attributes:

items (list of Product objects)

quantity (list of integers corresponding to the quantity of each product in the items list)

totalPrice (calculated based on product prices and quantities)

Order: Represents a confirmed purchase by a customer.

Attributes:

id (unique identifier)

customerID (foreign key referencing the Customer class)

orderDate (date/time of order placement)

products (list of Product objects included in the order)

Relationships:

Customer -has-a-> ShoppingCart: A customer can have one shopping cart at a time.

ShoppingCart -contains-> Product (many-to-many): A shopping cart can contain multiple products, and a product can be present in multiple shopping carts. (Quantity is tracked in the ShoppingCart class)

Customer -places-> Order (one-to-many): A customer can place multiple orders.

Order -contains-> Product (many-to-many): An order can contain multiple products, and a product can be part of multiple orders. (The quantity might be stored differently in the Order class depending on implementation)

Run down of execution:

Open Command Prompt and execute below command

```
ssh -p 1294 username@in-csci-rrpc05.cs.iupui.edu
```

Then enter the password

We will be redirected to a screen below

Go to the directory where all java files are available

In my case it is "Assignment3" directory

Type **rmiregistry**

```
System information as of Fri 19 Apr 2024 06:53:13 PM EDT

System load:  0.02           Temperature:    44.0 C
Usage of /:   4.0% of 228.17GB Processes:      190
Memory usage: 7%           Users logged in: 0
Swap usage:   0%           IPv4 address for eno1: 134.68.136.35

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

13 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '22.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Fri Apr 19 18:50:41 2024 from 47.231.160.0
raettadi@in-csci-rrpc05:~$ cd Assignment3
raettadi@in-csci-rrpc05:~/Assignment3$ rmiregistry
```

Now open another command prompt using same below command

```
ssh -p 1294 username@in-csci-rrpc05.cs.iupui.edu
```

Then enter the password

We will be redirected to a screen below

Clear all class files if any using below command

make -f makefile.mak clear

Now use below command to compile all java files using makefile

make -f makefile.mak

Then execute server using below command

make -f makefile.mak server

```
raettadi@in-csci-rrpc05: ~/As X + v
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

13 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '22.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Fri Apr 19 18:53:14 2024 from 47.231.160.0
raettadi@in-csci-rrpc05:~$ cd Assignment3
raettadi@in-csci-rrpc05:~/Assignment3$ cd store
raettadi@in-csci-rrpc05:~/Assignment3/store$ make -f makefile.mak clean
rm -f *.class
raettadi@in-csci-rrpc05:~/Assignment3/store$ make -f makefile.mak
javac -g CustomerService.java Administrator.java CustomerServiceImpl.java Client.java Order.java Customer.java FrontController.java OrderStatus.java FrontControllerImpl.java Server.java ProductService.java ShoppingCart.java AdministratorService.java ProductServiceImpl.java Product.java AdministratorServiceImpl.java
raettadi@in-csci-rrpc05:~/Assignment3/store$ cd ..
raettadi@in-csci-rrpc05:~/Assignment3$ make -f store/makefile.mak server
java -classpath . store.Server
FrontController server started.
New customer created: John
Customer updated: John
Product added: TShirt
Product updated: TShirt
```

Now open another command prompt using same below command
ssh -p 1294 username@in-csci-rrpc06.cs.iupui.edu
Then enter the password
We will be redirected to a screen below

Then execute client using below command
make -f makefile.mak client

Step 1:

I have logged in as Administrator using following credentials

Email: admin@example.com

Password: admin123

Now he has been provided with functionalities to add customers, update and delete and also to add products, update and delete them.


```

*** System restart required ***
Last login: Fri Apr 19 16:28:22 2024 from 104.241.40.161
raettadi@in-csci-rrpc06:~$ cd Assignment3
raettadi@in-csci-rrpc06:~/Assignment3$ make -f store/makefile.mak client
java -classpath . store.Client
Enter your email:
admin@example.com
Enter your password:
admin123
Administrator login successful
Enter customer name:
John
Enter customer email:
John@example.com
Enter customer password:
john123
Enter customer shipping address:
123 Street
Do you want to update customer details? (yes/no)
no
Do you want to remove customer John? (yes/no)
no
Customer removal cancelled.
Enter product name:
TShirt
Enter product description:
CalvinKlein
Enter product price:
50
Enter product quantity in stock:
100
Do you want to update product details? (yes/no)
no
Do you want to remove product TShirt? (yes/no)
no
Product removal cancelled.
raettadi@in-csci-rrpc06:~/Assignment3$ |

```

Step 2:

I have logged in as Customer using following credentials

Email: customer@example.com

Password: customer123

Now he has been provided with functionalities to show all available products and add products to cart and purchase them.

```

raettadi@in-csci-rrpc06:~/Assignment3$ make -f store/makefile.mak client
java -classpath . store.Client
Enter your email:
customer@example.com
Enter your password:
customer123
Customer login successful
In store
Available Products:
1 - TShirt - CalvinKlein - $50.0
2 - Trousers - UnderArmour - $80.0
Enter product ID to add to cart (0 to finish):
1
Enter quantity:
1
Product added to cart.
Order placed successfully! Thank you for shopping with us, !

```

Step 3: Create the JAR file

```
jar cvf Store.jar store/*.class
```

```
raettadi@in-csci-rrpc05:~/Assignment3/store$ cd ..
raettadi@in-csci-rrpc05:~/Assignment3$ jar cvf Store.jar store/*.class
added manifest
adding: store/Administrator.class(in = 1732) (out= 692)(deflated 60%)
adding: store/AdministratorService.class(in = 607) (out= 315)(deflated 48%)
adding: store/AdministratorServiceImpl.class(in = 4687) (out= 2060)(deflated 56%)
adding: store/Client.class(in = 7356) (out= 3612)(deflated 50%)
adding: store/Customer.class(in = 2591) (out= 1054)(deflated 59%)
adding: store/CustomerService.class(in = 444) (out= 268)(deflated 39%)
adding: store/CustomerServiceImpl.class(in = 2387) (out= 1270)(deflated 46%)
adding: store/FrontController.class(in = 392) (out= 252)(deflated 35%)
adding: store/FrontControllerImpl.class(in = 3074) (out= 1518)(deflated 50%)
adding: store/Order.class(in = 2430) (out= 1060)(deflated 56%)
adding: store/OrderStatus.class(in = 1092) (out= 613)(deflated 43%)
adding: store/Product.class(in = 2226) (out= 923)(deflated 58%)
adding: store/ProductService.class(in = 295) (out= 207)(deflated 29%)
adding: store/ProductServiceImpl.class(in = 993) (out= 574)(deflated 42%)
adding: store/Server.class(in = 1170) (out= 671)(deflated 42%)
adding: store/ShoppingCart.class(in = 1336) (out= 644)(deflated 51%)
raettadi@in-csci-rrpc05:~/Assignment3$
```